Repository Governance Decision Guide

Forty-one decisions that make repository-health assessment repeatable, automatable, and accountable

> **Purpose.** Give accountable leaders a finite, explainable set of choices that lets evidence collection become repeatable without turning every assessment into a new interview.

Document field	Value
Version	0.1.0-draft
Status	Provisional decision-support guide
Accountable standard owner	Enterprise Architecture
Prepared for	Repository owners, engineering, security, operations, risk, and architecture
Prepared	2026-08-09

Use boundary: this guide helps governance bodies make and record policy choices. It does not itself approve a choice, configure GitHub, accept risk, or prove that a control operated.

Executive purpose

A health assessor can observe a branch, a workflow run, a release, or a deployment. It cannot decide what the organization intends those facts to mean. For example, automation can report that a check is not required, but leadership must decide which checks are authoritative; it can find a release branch, but the owner must decide whether that branch is supported; and it can identify a failed control, but only an authorized risk owner can approve a temporary waiver.

The decisions in this guide create that missing contract. Once approved and recorded, most recurring proof can come from Git, GitHub, CI/CD, artifact, deployment, security, and operational systems rather than from a person completing a questionnaire. The objective is not to mandate one branching method. It is to make the chosen method explicit, observable, and governable.

Use this guide to produce three outcomes:

1. an approved repository contract that says what is in scope and how the repository is intended to operate;
2. authoritative records and system links that automation can read without inventing intent; and
3. named decision owners, review dates, and escalation routes for questions that cannot be settled by technical evidence alone.

The recommendations are defaults for discussion, not silent additions to 0.1.0-draft. A different value is acceptable when it still satisfies the governing outcome, is approved by the right authority, and leaves inspectable evidence. Unless an external requirement is cited explicitly, numeric cadences, retention periods, response targets, and waiver limits are RH 0.1.0-draft proposals—not universal industry mandates; stricter enterprise, legal, regulatory, contractual, or safety rules prevail.

How to use this guide in a workshop

Bring the repository owner, a technical maintainer, the relevant release or service owner, and representatives for security, operations, and portfolio governance when those functions apply. For Elevated or Critical repositories, include an independent reviewer and the delegated risk acceptor.

Work through the decisions in order because later answers depend on earlier classification:

1. Decide the foundation: scope, Main, type, lifecycle, risk, ownership, units, and critical references.
2. Decide the change contract: methodology, workflow, validation, checks, review, and bypass.
3. Decide the delivery and support contract: production correspondence, current outputs, release authority, support, security, recovery, evidence, and assessment ownership.
4. Resolve conditional controls only when their triggering fact applies.
5. Use formal dispositions for N/A, equivalence, waivers, accepted risk, appeals, and issuance; never hide these decisions in free-form notes.

For every item, record the selected value, accountable approver, decision date, effective date, review date or trigger, rationale, and stable evidence links. Mark unresolved items Unknown; do not select the most favorable value to improve a score. At the end of the workshop, reconcile the human-readable decision record with the machine-readable repository declarations and the actual GitHub settings.

Decision, implementation, and evidence are different

- A **decision** states the intended outcome and who accepts accountability for it. Example: “All changes to Main require the final revision to pass the validation and link-check jobs.”
- An **implementation** is the mechanism chosen to carry out the decision. Example: a GitHub ruleset requires those two checks.
- **Evidence** is the current or historical record showing what is configured and what actually happened. Example: the effective ruleset plus completed check runs for accepted revisions.

A policy file alone normally establishes only a declaration. A ruleset may establish current configuration. Change, build, release, deployment, or recovery history is needed to demonstrate operation and sustained effectiveness. The E0–E4 assurance model preserves these distinctions.

Risk acceptance is also distinct from conformance. Approving a risk does not turn an Unmet or Unknown control into Met, restore missing evidence, or lift a foundational-gate cap.

Decision authority model

Use durable organizational roles rather than relying only on named individuals. A person may hold several roles in a small organization, but an Elevated or Critical repository should keep assessment and risk acceptance independent.

- **Enterprise Architecture** owns the standard, thresholds, recognized cross-repository equivalences, disputed policy interpretations, and publication of a standard version.
- **Repository or technical owner** is accountable for the repository contract, validation path, maintainability, and remediation.
- **Business, service, or product owner** defines supported outcomes, consumers, service boundaries, and business impact.
- **Repository administrator and CI/platform owner** configure and preserve effective source-control and automation enforcement.
- **Release or operations owner** defines release, promotion, current-output, recovery, and operational-readiness decisions.
- **Security owner** defines security reporting, secret response, CI/CD trust, vulnerability handling, and security-sensitive approvals.
- **Portfolio owner** establishes authoritative-repository, lifecycle, archive, and retirement records.

- **Assessment lead and assessor** define assessment scope, collect evidence, make unadjudicated ratings, preserve limitations, and issue the assessment under the approved standard.
- **Risk acceptor** accepts time-bounded residual risk within delegated authority; this role does not change the observed rating.
- **Adjudication chair or control-domain owner** resolves assessment disputes, escalating precedent-setting decisions to Enterprise Architecture.

Recording-location labels

Each decision distinguishes three kinds of location:

- **Platform/industry** — a filename, location, or settings surface documented by a platform or established specification. Examples include .github/CODEOWNERS, SECURITY.md, CONTRIBUTING.md, .github/workflows/, GitHub rulesets, and protected environments.
- **RH recommendation** — a path or schema selected by this Repository Health project for machine readability or consistent review. Examples include .github/repository-health.toml, .github/repository-health-production.json, and .github/repository-health-evidence-policy.json. These are not universal industry filenames.
- **External system of record** — the authoritative catalog, identity, deployment, artifact, security, risk, incident, records-management, or change system. Keep the durable record there and place only a stable identifier or link in the repository when copying the record would create stale or sensitive data.

Enforcement truth belongs in the effective platform or delivery system, not only in prose. Repository documentation explains the contract and points to that authority.

Summary index

Foundation decisions

1. Assessment scope — What repository, units, time, systems, and exclusions are assessed.
2. Canonical Main — Which exact ref performs the Main role.
3. Repository type — Which type-specific health contract applies.
4. Lifecycle state — Whether the repository is active, supported, experimental, mirrored, archived, retired, or unknown.
5. Risk tier and rationale — Baseline, Elevated, or Critical assurance expectations.
6. Accountable ownership — Who owns technical, business, support, and security outcomes.
7. Deployable/publishable units — Which independently released or operated units exist.
8. Production-critical references — Which refs can determine production, release, or preserved state.

Workflow and change decisions

9. Intended Git methodology — The declared workflow profile and any hybrid components.
10. Workflow rules — The observable integration, branch, release, promotion, hotfix, and history rules.
11. Validation contract — What a clean checkout must validate for each change and unit.
12. Authoritative checks — Which exact checks must pass on the accepted revision.
13. Review and separation-of-duty policy — Who must independently approve which changes.
14. Emergency/bypass policy — How privileged exceptions are constrained, logged, and reviewed.

Production, support, and assessment decisions

15. Production-correspondence contract — How current outputs correspond to Main.
16. Current-output definition — What running, published, distributed, or supported state counts as current.
17. Release and promotion authority — Who or what may authorize movement into a release or production target.
18. Support boundary — Which versions, consumers, environments, and periods receive support.
19. Security-reporting policy — How vulnerabilities are reported privately, acknowledged, and handled.
20. Rollback/withdrawal policy — How each unit is reverted, restored, superseded, or withdrawn.
21. Recovery exercise frequency — How often the recovery method must be demonstrated.
22. Evidence retention — How long assessment, release, deployment, and operational evidence remains available.
23. Assessment ownership — Who assesses, reviews, issues, and schedules reassessment.

Conditional-control decisions

24. Critical-path classification — Which paths need named qualified owners and continuity.
25. Supported release-line policy — Which release or maintenance lines remain supported and when they end.
26. Dependency/SBOM policy — Which dependencies and released artifacts need inventory and SBOM evidence.
27. Vulnerability-remediation targets — How quickly verified findings must be resolved by severity.
28. Secret-response policy — How exposed credentials are prevented, revoked, investigated, and recorded.
29. CI/CD trust policy — How automation identities, permissions, third-party code, untrusted inputs, and secrets are constrained.
30. Toolchain and technical-obligation policy — How runtimes, build images, generators, debt, and maintenance dates are governed.
31. Generated or binary artifact disposition — What may be committed and what provenance or regeneration evidence is required.

32. Public-contribution policy — Whether and how outside contributors may propose changes and receive support.
33. DORA applicability — Whether reliable product-level delivery outcomes can be reported separately.
34. Portfolio authority — Which repository is authoritative and how mirrors, forks, templates, and successors relate.
35. Archive/retirement disposition — How an inactive repository is made safe, discoverable, and restorable where required.

Formal-disposition decisions

36. N/A determination — Whether an outcome truly does not apply, with evidence and approval.
37. Equivalent-control approval — Whether an alternate mechanism provides equal or stronger outcomes and evidence.
38. Temporary waiver — Whether an unmet requirement is accepted for a short, expiring remediation period.
39. Accepted vulnerability or operational risk — Who accepts a documented residual risk without changing the rating.
40. Assessment appeal or interpretation — How facts, applicability, evidence, arithmetic, and policy disputes are adjudicated.
41. Publication/issuance decision — Whether a result is complete and authorized for its intended audience.

Foundation decisions

1. Assessment scope

Term. The exact repository, remote, revision, units, evidence window, connected systems, and exclusions covered by one assessment.

Decision question. What is the bounded subject of this assessment, and what is explicitly outside it?

What must be decided. Identify the canonical repository URI; the exact Main revision or assessment cutoff; every deployable or publishable unit; repository-wide and unit-specific boundaries; the observation window; connected evidence systems; inaccessible systems; and any exclusion. A monorepo-wide label cannot hide an omitted unit, and an exclusion cannot be used as an informal N/A.

Why it matters. A score is meaningful only when readers can tell which assets and evidence it represents. Ambiguous scope produces incomplete denominators, makes comparison unreliable, and can conceal a failed or Unknown foundational gate.

Acceptable values. A whole-repository assessment; a whole repository plus explicitly enumerated units; or a bounded unit assessment that is clearly marked as not representing repository-wide health. Use a

trailing 90-day event window, extending to 365 days for low samples, unless a control specifies another window. Record exclusions as `none` or as named boundaries with rationale and effect.

> **Recommended default.** Assess the canonical remote, exact Main `HEAD`, all active deployable or publishable units, current configuration, and the standard's W90/W365 evidence windows. Treat inaccessible applicable evidence as Unknown rather than excluding it.

Decision authority. Assessment lead, with factual confirmation from the repository owner; unit and operational boundaries should also be confirmed by the service, product, or release owner.

Record in. **RH recommendation:** repository identity, units, and classification in `.github/repository-health.toml`; immutable assessment scope and cutoff in the issued assessment JSON or workbook. **External system of record:** service catalog or application inventory for organizational scope, linked by stable ID.

Evidence and review. Preserve the exact repository URL, assessed SHA, cutoff time, window, unit inventory, source-system access results, and exclusions. Reconfirm before every issued assessment and whenever a unit, canonical remote, or delivery target changes.

Source basis. SRC-001; Repository Health Standard, scope; Measurement Dictionary, common rules.

2. Canonical Main

Term. **Main** is the one canonical, integrated line of record; the literal branch name may be different.

Decision question. Which exact branch or equivalent ref is accountable for the integrated, releasable source and production lineage?

What must be decided. Name the full ref that performs Main, state whether it is also the default branch, and explain any difference. Define how release, hotfix, maintenance, environment, or fork changes return to that line. If no ref satisfies the Main contract, record the condition as Unknown or nonconforming rather than assigning a convenient branch.

Why it matters. Buildability, controlled changes, production reachability, and critical-ref protection all depend on a single unambiguous integration role. A default branch name or `main` label is a candidate, not proof.

Acceptable values. A normal branch such as `refs/heads/main`, `refs/heads/master`, or another organization-approved integration branch; an equivalent Git ref only when it demonstrably supplies the same immutable history and integration role. A separate default branch is acceptable when the distinction and safeguards are documented.

> **Recommended default.** Use the protected default branch named `main` as Main. Require every production-only fix to be reconciled into Main through the declared change path.

Decision authority. Repository technical owner and maintainer lead; repository administrator confirms that host settings implement the decision.

Record in. RH recommendation: repository.main_branch in .github/repository-health.toml. **Platform/industry:** default-branch and effective ruleset settings in GitHub. Explain a non-default Main and reconciliation path in CONTRIBUTING.md or GOVERNANCE.md.

Evidence and review. Use the exact ref, commit graph, default-branch setting, protection rules, release ancestry, and observed merge history. Review when changing the default branch, methodology, release topology, or authoritative remote.

Source basis. SRC-001; Git definitions and reachability cited by the Main contract; SRC-010 for GitHub ruleset behavior.

3. Repository type

Term. The supported outcome category that determines what "buildable, validated, and releasable" means for this repository.

Decision question. Which repository type best describes the asset's intended organizational purpose, and do any units require a different functional type?

What must be decided. Select one primary type, record material secondary types, and give each monorepo unit its functional type. Base the decision on supported use and operating outcome, not file extensions or the presence of incidental documentation, examples, or test fixtures.

Why it matters. A documentation repository, package, GitOps repository, mirror, and archived application need different validation and production interpretations. Wrong typing creates false failures, false N/A decisions, and misleading comparisons.

Acceptable values. Deployable application, Library/package, Monorepo, Infrastructure-as-code/GitOps, Data/analytics/model, Documentation/content, Template/scaffold, Sandbox/experimental, Mirror/fork, Archived/retired, or Unknown. Mixed repositories may add secondary and unit-specific types without inventing new top-level labels.

> **Recommended default.** Choose the narrowest canonical type that matches the supported output. Use Unknown until ownership and consumer evidence resolve a genuinely ambiguous case; do not default all repositories to deployable applications.

Decision authority. Repository owner and service/product owner, confirmed by the assessment lead; the portfolio owner resolves conflicts with enterprise inventory.

Record in. RH recommendation: repository.type and the unit inventory in .github/repository-health.toml; unit-specific types in the associated production-identity record or assessment scope. **External system of record:** service or portfolio catalog, with a stable link from the repository.

Evidence and review. Use purpose documentation, consumers, release and deployment records, package or publication targets, and portfolio metadata. Review when supported purpose, operating model, or independently released units change.

Source basis. SRC-001; Classification Guide, repository types; type-specific buildability.

4. Lifecycle state

Term. The repository's current maintenance and support posture, distinct from its recent activity level.

Decision question. Is this repository actively changing, stably supported, experimental, mirrored, archived, retired, or not yet classifiable?

What must be decided. Select the current lifecycle state, its effective date, expected support or maintenance behavior, and any successor or upstream. Distinguish deliberate low activity from abandonment and distinguish an archive declaration from merely having no recent commits.

Why it matters. Lifecycle determines which build, support, vulnerability, activity, and retirement expectations apply. It prevents healthy stable repositories from being penalized and prevents abandoned production assets from being mislabeled as stable.

Acceptable values. `Active`, `Stable-supported`, `Experimental`, `Mirrored`, `Archived`, `Retired`, or `Unknown`. Use `Archived` or `Retired` only with an intentional disposition and the controls required by RLP-03.

> **Recommended default.** Use `Active` for repositories that still accept supported changes or create current outputs; use `Stable-supported` only with an identified owner and current support intent. Select `Unknown` when neither intent nor disposition can be evidenced.

Decision authority. Business/service or product owner with the repository owner; portfolio owner approves archive or retirement status.

Record in. RH recommendation: `repository.lifecycle` in `.github/repository-health.toml`. **Platform/industry:** GitHub archive mode when archived. Put the visible status, date, support boundary, and successor in `README.md` or `GOVERNANCE.md`. **External system of record:** portfolio or lifecycle catalog.

Evidence and review. Owner confirmation, portfolio state, support records, current consumers/outputs, archive settings, credentials, automation, and successor links. Review at least at every assessment and on support, archive, retirement, or ownership changes.

Source basis. SRC-001; Classification Guide, stable low activity; controls RLP-01 and RLP-03 in the Control Catalog.

5. Risk tier and rationale

Term. The consequence-based assurance tier used to scale control strength, evidence, freshness, and independence.

Decision question. What is the highest credible impact of failure, compromise, or unrecoverable change for this repository or any material unit?

What must be decided. Select a tier for each material unit, assign the repository-wide minimum from the highest unit tier for shared controls, and record the business, safety, regulatory, data, privilege, availability, blast-radius, and recovery facts supporting the choice. Lack of stronger evidence is not a reason to lower the tier.

Why it matters. The same repository practice can be adequate for a low-impact template and inadequate for privileged critical infrastructure. The tier governs assurance expectations without treating repository importance or team performance as risk.

Acceptable values. `Baseline`, `Elevated`, or `Critical`. When evidence supports two plausible tiers, use the higher provisionally and record what evidence or owner decision will resolve it.

> **Recommended default.** Use Baseline only when no material safety, regulatory, privileged-access, sensitive-data, broad-consumer, financial, operational, or difficult-recovery factor applies. Use Elevated for production services, shared platforms, sensitive data, or material operational impact; use Critical for severe safety, systemic, regulated, privileged, or recovery consequences.

Decision authority. Business/service owner and repository owner propose; the delegated risk or compliance authority confirms Elevated and Critical classification. Enterprise Architecture owns the tier policy, not each repository's business facts.

Record in. RH recommendation: `repository.risk_tier` in `.github/repository-health.toml` and rationale in a linked decision record under `docs/decisions/` when it does not fit machine-readable metadata. **External system of record:** enterprise risk, service criticality, data classification, or business-impact record.

Evidence and review. Retain the unit-level impact factors, approving roles, evidence links, date, and reassessment triggers. Review at least annually and on material changes to consumers, data, privilege, availability, regulation, deployment target, or recovery difficulty.

Source basis. SRC-001; Repository Health Standard, risk tiers; Classification Guide, risk checks.

6. Accountable ownership

Term. Durable roles accountable for the repository's technical condition, business or service outcome, support route, and security response.

Decision question. Which active team and accountable roles own the repository, its outputs, support, security response, and remediation?

What must be decided. Name the technical owner; business, service, or product owner where applicable; support owner; security owner; and unit owners when responsibilities differ. Define

escalation and continuity when one person is unavailable. CODEOWNERS identifies review responsibility for paths but does not by itself establish business accountability.

Why it matters. Automation can find files and teams, but it cannot infer who accepts responsibility. Missing or personal-only ownership creates orphaned risk, slow response, and unreviewed critical paths.

Acceptable values. A durable team or organizational role backed by active members; a named accountable role plus monitored group route; or an inherited enterprise owner with a resolvable repository-level link. A personal address alone is not a durable default. Elevated and Critical critical paths should have two qualified people or an approved continuity plan.

> **Recommended default.** Record one durable technical-owner team, one business/service owner where relevant, monitored support and security routes, and path-level CODEOWNERS. Use team aliases and role records while allowing current people to be resolved from the identity system.

Decision authority. Business/service owner assigns accountability; engineering or maintainer leadership assigns technical owners; security and support functions approve their routes.

Record in. RH recommendation: `repository.owner` in `.github/repository-health.toml`. **Platform/industry:** `.github/CODEOWNERS` is GitHub's preferred protected location, although root and `docs/` are supported; `SUPPORT.md`, `SECURITY.md`, and `GOVERNANCE.md` are recognized community files. **External system of record:** service catalog and identity/group directory.

Evidence and review. Verify team existence, active membership, CODEOWNERS coverage, support/security route delivery, and owner confirmation. Review on team reorganization, personnel loss, critical-path changes, and at the tier-specific route-verification interval.

Source basis. SRC-001; controls OWM-01, OWM-02, and DCR-03; SRC-004 and SRC-018; GitHub's CODEOWNERS documentation.

7. Deployable/publishable units

Term. Each independently deployed, published, applied, distributed, or supported output that needs its own identity and delivery evidence.

Decision question. What distinct units can change, release, deploy, publish, roll back, or end support independently?

What must be decided. Assign a stable unit ID, owner, functional type, source boundary, validation path, target or consumer, current-output identity, risk tier when different, and recovery or withdrawal path. Shared source commits do not remove the need for separate release and target records.

Why it matters. A repository-wide green build or release label can hide a failed service, package, model, site, or environment. Per-unit identity prevents evidence for one unit from incorrectly satisfying another.

Acceptable values. One repository-wide unit; several named units in a monorepo; a publishable content or package unit; or no active unit only when the repository type and current-output decision support that conclusion. Unit IDs should be stable, unique, and safe for machine use.

> **Recommended default.** Create one unit for every independently versioned or promoted output, even when several units use the same Git SHA. Do not use a catch-all monorepo unit when release, owner, target, or rollback boundaries differ.

Decision authority. Repository owner with service/product and release owners; assessment lead confirms that all in-scope units are represented.

Record in. RH recommendation: `repository.deployable_units` in `.github/repository-health.toml` and full selectors in `.github/repository-health-production.json`. Keep the evidence selection contract in `.github/repository-health-evidence-policy.json`. **External system of record:** service catalog, registry, deployment, publication, or model catalog.

Evidence and review. Compare repository structure, build graph, releases, registries, deployments, publications, consumers, and portfolio records. Review when adding, splitting, merging, renaming, transferring, or retiring a unit.

Source basis. SRC-001; Repository Health Standard, deployable units; Production Identity.

8. Production-critical references

Term. Main and every branch, tag, environment ref, GitOps ref, or equivalent reference capable of determining a current output or preserving a supported state.

Decision question. Which references could authorize, select, or materially change production, publication, release, maintenance, or retained supported history?

What must be decided. Enumerate each ref or rule pattern, its purpose, owner, allowed update path, mutability, deletion and rewrite policy, required checks, bypass roles, and end-of-life condition. Include immutable release tags and environment or GitOps refs when they control delivery.

Why it matters. Protecting only Main is insufficient when an unprotected tag, release branch, environment branch, or desired-state ref can change what users receive. An unclassified active ref makes protection coverage indeterminate.

Acceptable values. Main only when no other ref can influence or preserve a current output; Main plus release or maintenance branches; Main plus protected immutable tags; Main plus environment/GitOps refs; or a documented custom set. Work branches are not production-critical merely because they exist.

> **Recommended default.** Treat Main, every supported release/maintenance branch, production environment or GitOps ref, and every release tag used as an immutable output identity as critical. Deny deletion and force updates by default and require the declared acceptance path.

Decision authority. Repository administrator and technical owner, with release/operations and security owners for refs that determine production.

Record in. Platform/industry: effective GitHub rulesets, tag rules, environments, and access controls are the enforcement system of record. **RH recommendation:** declare Main and production selectors in .github/repository-health.toml and .github/repository-health-production.json; explain ref purposes in GOVERNANCE.md or CONTRIBUTING.md.

Evidence and review. Preserve the complete ref inventory, effective overlapping rules, update and bypass history, production selectors, and EOL records. Review continuously through assessment and whenever release, promotion, support-line, or environment topology changes.

Source basis. SRC-001; control CGD-02; SRC-005, SRC-010, and SRC-013.

Workflow and change decisions

9. Intended Git methodology

Term. The declared workflow profile that best explains how work integrates, releases, promotes, and returns fixes to Main.

Decision question. Which supported methodology—or intentional combination—does the organization expect this repository to follow?

What must be decided. Select the declared primary profile, identify material secondary profiles, describe custom or per-unit components, and state where the method applies. Keep the declared profile separate from the assessor's observed profile and record contradictions rather than overwriting either.

Why it matters. Branch names alone do not explain intent. Declaring the method makes branch lifetime, merge direction, release source, promotion, hotfix, and cleanup behavior testable without awarding or deducting points for the methodology name itself.

Acceptable values. Trunk-based, GitHub Flow/short-lived feature branches, GitFlow, Environment-branch flow, Release train/multi-version maintenance, GitOps promotion, Fork/integration-manager, Direct gated trunk, Custom/hybrid, Unknown, or Unclassified. Custom/hybrid is valid when decomposed by axis and unit; it is not a hiding place for uncertainty.

> **Recommended default.** Use the simplest profile that accurately explains observed integration and release behavior. For a short-lived branch and pull-request flow into deployable Main, select GitHub Flow/short-lived feature branches; use Custom/hybrid when, for example, GitHub Flow ingress feeds GitOps promotion.

Decision authority. Maintainer lead and repository technical owner, with release/operations owners for promotion behavior. The assessor independently records the observed profile and confidence.

Record in. RH recommendation: repository.methodology, repository.observed_methodology, confidence, evidence IDs, contradictions, and methodology_axes in .github/repository-health.toml. Describe contributor-facing rules in CONTRIBUTING.md or GOVERNANCE.md.

Evidence and review. Compare the declaration with commit graphs, pull requests, refs, releases, deployments, reconciler behavior, and the nine workflow axes. Review when branch topology, release source, promotion mechanism, support lines, or repository topology changes.

Source basis. SRC-001; Classification Guide, methodology profiles; methodology references SRC-012, SRC-014 through SRC-017, and SRC-021.

10. Workflow rules

Term. The observable rules governing change ingress, branch purpose and lifetime, integration cadence, release source, promotion, parallel support, hotfix propagation, control placement, repository topology, and history.

Decision question. What must contributors and automation do from proposed change through integration, release, promotion, maintenance, and cleanup?

What must be decided. Complete every workflow axis, define allowed branch/ref classes, merge or submission paths, release and backport directions, emergency-fix reconciliation, branch EOL, history rewrite, and per-unit variations. Define a cadence-relative branch SLO where stale-ref assessment applies.

Why it matters. A methodology label is too coarse to score or automate. Explicit axes let the assessor distinguish an intentional long-lived release branch from unmanaged drift and test whether actual behavior matches the declared design.

Acceptable values. One Main line, Main plus develop, multiple environment lines, multiple supported version lines, or hierarchical integrators; direct gated submission, short-lived branches, forks, or controlled promotion; Main/tag, release branch, environment branch, artifact, or GitOps ref as release source. History may use merge commits, squash, rebase, cherry-pick/backport, or another auditable rule consistent with the profile.

> **Recommended default.** Define all nine axes in the classification guide, use short-lived change refs for routine work unless the selected profile needs otherwise, forbid unexplained production-only commits, and require a documented return path to Main for hotfixes and backports.

Decision authority. Maintainer lead and repository technical owner; release owner approves release/promotion and supported-line behavior; repository administrator confirms enforceability.

Record in. Platform/industry: CONTRIBUTING.md is a GitHub-discoverable contributor guide; .github/workflows/ and effective rulesets implement parts of the flow. **RH recommendation:** record

normalized axes in .github/repository-health.toml; put detailed workflow and diagrams in CONTRIBUTING.md or GOVERNANCE.md.

Evidence and review. Use the declared axes, branch/ref inventory, merge bases, accepted changes, releases, promotions, backports, reconciliation history, and cleanup records. Review after workflow exceptions recur or any topology/cadence change.

Source basis. SRC-001; Classification Guide, workflow axes; controls CGD-05, DCR-02, and RLP-02.

11. Validation contract

Term. The version-controlled, non-interactive process that validates or creates the repository's deliverable from a clean checkout.

Decision question. What must succeed for the exact Main revision and each proposed change or affected unit to be considered buildable, validated, and releasable for type?

What must be decided. Define commands or workflows, declared inputs, dependency restoration, environment/toolchain, change classes, applicable validation categories, affected-unit rules, release checks, and the expected terminal result. Cover code, IaC, data, content, templates, mirrors, and archives according to their type-specific outcome.

Why it matters. “The build passed” is meaningless until the organization defines what the build must establish. The validation contract is the basis of G-01 and prevents a health action from substituting its own execution for the repository's real checks.

Acceptable values. Compile/package and test; parse, policy-check, and plan; render and validate links/metadata; reproduce a data/model artifact; instantiate a representative template; compare a mirror to upstream; or validate archival readability and disposition. Different change classes or units may use different complete category sets.

> **Recommended default.** Provide one documented clean-checkout entry point that invokes all required categories, run it automatically on the final proposed revision and exact Main revision, and define affected-unit selection without allowing an aggregate status to mask an unvalidated unit.

Decision authority. Repository technical owner with quality and CI owners; type-specific data, infrastructure, security, or documentation owners approve their required categories.

Record in. Platform/industry: version-controlled build files and .github/workflows/*.yml; README.md or CONTRIBUTING.md for the supported command. **RH recommendation:** reference applicable categories in .github/repository-health-evidence-policy.json and identify authoritative checks in .github/repository-health.toml.

Evidence and review. Clean-checkout instructions, exact workflow/configuration, dependency and toolchain declarations, proposed-change runs, exact-Main runs, unit coverage, and terminal results. Review when outputs, dependencies, change classes, release requirements, or tools change.

Source basis. SRC-001; controls BTC-01 through BTC-03; type-specific buildability; SRC-003 and SRC-009.

12. Authoritative checks

Term. The exact, identity-bound status checks whose successful result on the final accepted revision establishes the required validation outcome.

Decision question. Which checks are mandatory, what produces them, and how are similarly named or stale results prevented from satisfying the rule?

What must be decided. Name each required check by stable workflow/job or application identity, bind it to the expected GitHub App or trusted producer where supported, define which change classes and units it covers, require success on the final revision, and state strictness or merge-queue behavior. Do not list the repository-health assessment itself unless it performs the repository's actual validation.

Why it matters. A check name alone can collide, be produced by an unexpected source, or refer to an earlier revision. Exact authoritative identities connect the validation contract to effective acceptance enforcement.

Acceptable values. A minimal set of independent build, test, content, policy, or security checks; a matrix whose aggregate cannot hide a failed required job; merge-queue checks on the queued revision; or equivalent pre-submit admission for direct gated trunk. Manual evidence is not an authoritative automated check.

> **Recommended default.** Require the smallest complete set that establishes the validation contract, bind each check to its trusted producer, and require it to pass on the exact final revision before acceptance. Keep the health-assessment check separate from buildability evidence.

Decision authority. Technical and quality owners select outcomes; CI owner and repository administrator approve producer identity and enforcement.

Record in. **Platform/industry:** GitHub rulesets are the enforcement system of record; workflow definitions live in `.github/workflows/`. **RH recommendation:** declare the controlled check identities in `repository.authoritative_checks` in `.github/repository-health.toml` and validation categories in `.github/repository-health-evidence-policy.json`.

Evidence and review. Effective required-check settings, expected source application, final-revision check suites, accepted-change history, bypasses, and rename/migration records. Review whenever a workflow, job name, GitHub App, merge mode, or validation category changes.

Source basis. SRC-001; controls BTC-02, BTC-03, and CGD-04; SRC-010 for required-status-check and expected-source behavior.

13. Review and separation-of-duty policy

Term. The rule that determines when a trusted person independent of the author must approve the final accepted revision and which roles may do so.

Decision question. Which changes require independent review, how many approvals are required, and who is disqualified by authorship, ownership conflict, or insufficient authority?

What must be decided. Set review requirements by risk, path, ref, and change type; define eligible reviewer teams; require review of the final revision; decide whether stale approvals are dismissed; protect CODEOWNERS; and define continuity for small teams without silently self-approving.

Why it matters. Independent review reduces accidental and malicious change risk, but a nominal pull request does not provide separation if the author can approve, dismiss, or bypass their own final change.

Acceptable values. Baseline self-maintainer path with all checks and explicit risk rationale when no second trusted person is available; one independent approval for Elevated; a declared two-party policy for Critical; risk- or path-based CODEOWNER review; or an equivalent pre-submit review mechanism with the same independence and auditability.

> **Recommended default.** Require at least one independent trusted approval for production, security, release, infrastructure, and other critical paths; apply one independent final-revision approval to Elevated repositories and the declared two-party policy to Critical repositories. Keep the author from satisfying the independent approval.

Decision authority. Engineering manager or maintainer lead with the repository owner; security owner approves security-sensitive reviewer boundaries. Any non-profile equivalent for Elevated/Critical requires Architecture disposition.

Record in. Platform/industry: .github/CODEOWNERS and effective GitHub rulesets/review settings. Explain eligibility, final-revision, stale-approval, and small-team rules in GOVERNANCE.md or CONTRIBUTING.md. **External system of record:** identity teams and delegated approval authority.

Evidence and review. CODEOWNERS content and protection, effective review rules, pull-request approvals on final revisions, dismissal and bypass events, reviewer membership, and continuity plans. Review on owner/team changes and at every assessment for Elevated/Critical repositories.

Source basis. SRC-001; control CGD-03; SRC-004, SRC-005, and SRC-010; GitHub CODEOWNERS documentation.

14. Emergency/bypass policy

Term. The controlled path for bypassing, administering, or exceptionally changing a protected ref or delivery control when the normal path cannot be used.

Decision question. When may a control be bypassed, who may do it, what must be recorded, and how quickly must access and the resulting change be reviewed?

What must be decided. Define permitted scenarios, eligible break-glass roles, least-privilege access, approval or incident authority, required reason and ticket, affected controls/refs, logging, access expiry, post-use review, remediation, and reconciliation to Main. Decide whether any bypass is entirely prohibited.

Why it matters. An undocumented administrative path can nullify otherwise strong rules. A legitimate emergency path must preserve accountability without pretending the normal control operated.

Acceptable values. No bypass; narrowly scoped break-glass with prior approval; emergency use with documented incident authority and prompt retrospective review; or platform-admin action for rule repair. Permanent broad bypass lists, shared credentials, or unlogged force operations are not acceptable defaults.

> **Recommended default.** Deny routine bypass. Limit break-glass to named roles, require a stable incident/change ID and reason, log the exact actor and affected ref/control, expire temporary access automatically, reconcile changes to Main, and complete an independent post-use review before closure.

Decision authority. Repository administrator and security owner define the mechanism; service/release owner authorizes operational use; delegated risk acceptor approves residual risk. The user performing the bypass should not be the sole reviewer.

Record in. **Platform/industry:** GitHub ruleset bypass actors, environment reviewers, access controls, and audit log are authoritative. Describe the process in GOVERNANCE.md or an operational runbook. **External system of record:** incident/change ticket and privileged-access system; store only stable IDs in the assessment or repository decision record.

Evidence and review. Effective bypass configuration, identity membership, every use event, reason/ticket, approval or retrospective review, access expiry, reconciliation, and follow-up. Review after every use and whenever privileged roles or protection mechanisms change.

Source basis. SRC-001; control CGD-06 and gate G-03; SRC-010 and SRC-013 for platform controls.

Production, support, and assessment decisions

15. Production-correspondence contract

Term. The declared relationship between Main and the immutable source revision represented by each current deployed, published, applied, distributed, or supported output.

Decision question. For each unit, must current output equal Main, be a validated revision reachable from Main, come through a controlled production ref, or use an explicit per-unit combination?

What must be decided. Select one contract per unit; define the production selector and immutable output identity; state how Main reachability is proved; define acceptable Main-ahead lag; require intervening validation for Releasable-Main; define production-ref reconciliation; and state how drift becomes visible.

Why it matters. "Main represents production" does not require Main `HEAD` to be deployed, but it does require provable lineage. Without a contract, automation cannot distinguish intentional lag from source, artifact, or environment drift.

Acceptable values. `Exact-Main`, `Releasable-Main`, `Production-Ref`, `Per-unit combination`, `N/A` only through approved applicability, or `Unknown`. Mutable branch names, environment labels, semantic versions, or `latest` aliases alone are not sufficient identities.

> **Recommended default.** Use `Releasable-Main`: every current output maps to an immutable source revision reachable from Main, and every accepted Main revision between that source and assessed Main passed all authoritative checks. Use Exact-Main only when deployment is intentionally synchronized; use Production-Ref only with controlled promotion and reconciliation.

Decision authority. Release owner and service/product owner with the repository technical owner; security or platform owner confirms provenance and target controls for Elevated/Critical units.

Record in. RH recommendation: `repository.production_correspondence` in `.github/repository-health.toml` and per-unit selectors, source SHAs, artifact/content identities, and limitations in `.github/repository-health-production.json`. **External system of record:** deployment, registry, release, publication, or GitOps reconciler records.

Evidence and review. Exact production identity, artifact/content digest, source SHA, Main ancestry, authoritative-check history, terminal deployment/publication status, promotion authorization, and current target inventory. Review on every current-output change and when the delivery topology changes.

Source basis. SRC-001; Production Identity; controls SPI-02 and SPI-04; SRC-005 and SRC-011.

16. Current-output definition

Term. The rule for deciding which running, published, applied, distributed, or supported-consumer states must be represented in production traceability.

Decision question. What outputs are current and in scope now, including supported releases that are not the newest deployment?

What must be decided. Enumerate environments, channels, registries, sites, packages, models, datasets, infrastructure states, and supported consumer versions that count as current. Define selection by immutable IDs, not aliases. If claiming no current output, define the consumer and delivery discovery required and the approver/review date.

Why it matters. A "latest release" query can omit a supported prior line, regional deployment, package channel, or GitOps target. Conversely, historical artifacts alone should not force an active-production result for a safely retired repository.

Acceptable values. All successful production deployments; every published channel or package version still supported; every current GitOps target/ref; every active publication; a controlled exact set of units/targets; `approved-no-current-output`; or `archived-retired`. Empty API results and low activity do not by themselves prove no current output.

> **Recommended default.** Include every output that is deployed, published, applied, distributed, or promised support to a consumer, not just the newest. Use the active per-unit production profile with an exact selector; use no-output or archive profiles only with independent discovery and governance evidence.

Decision authority. Service/product and release owners; portfolio owner for archives; assessment lead verifies that the population is complete rather than accepting a self-declaration.

Record in. RH recommendation: `repository.has_current_output`, `repository.publishes_artifacts`, and unit IDs in `.github/repository-health.toml`; use the controlled unit profile and selector in `.github/repository-health-production.json`. **External system of record:** deployments, registries, package repositories, publications, service catalog, and consumer records.

Evidence and review. Complete current target/output inventories, supported-version policy, immutable identities, terminal status, consumer discovery, archive/no-output approval, and review dates. Re-evaluate at every assessment and whenever support or delivery state changes.

Source basis. SRC-001; M-SPI-02S in the Measurement Dictionary; inactive and no-output rules.

17. Release and promotion authority

Term. The roles, automation identities, approvals, and input constraints allowed to create a release or move an output into a production target.

Decision question. Who or what may authorize, execute, and verify release or promotion for each unit and target?

What must be decided. Define authorized source refs and immutable artifacts, human and automation identities, required approvals, environment protections, credential scope, concurrency, terminal success, separation of duties, and durable release/deployment record. Distinguish authorization from execution and verification.

Why it matters. Strong repository controls stop at the wrong boundary if an untrusted ref, mutable artifact, broad credential, or unreviewed actor can still alter production.

Acceptable values. Automated release from a protected tag or Main revision; protected-environment promotion; GitOps reconciliation from a controlled ref/path; scoped registry publisher; or documented manual promotion with durable approval and evidence. Elevated and Critical paths require stronger independence and inspectable identity.

> **Recommended default.** Permit only immutable artifacts or authorized refs produced by the validated source path; use a dedicated least-privilege automation identity, protected target/environment, explicit authorization, serialized conflicting operations where needed, and independent post-deployment verification.

Decision authority. Release/platform owner and service/product owner; security owner approves credentials, identities, and trust boundaries; delegated business authority approves production promotion where required.

Record in. Platform/industry: protected GitHub environments, rulesets, workflow permissions, and deployment history. **RH recommendation:** source/target selectors in `.github/repository-health-production.json` and evidence statements selected through `.github/repository-health-evidence-policy.json`. **External system of record:** deployment controller, registry, GitOps reconciler, change, and identity systems.

Evidence and review. Effective inputs, identities, permissions, reviewers, concurrency controls, artifact/source bindings, deployment records, terminal status, and bypass history. Review on identity, target, workflow, artifact format, or approval-policy change.

Source basis. SRC-001; controls RRO-01 and RRO-03; SRC-005, SRC-010, and SRC-011.

18. Support boundary

Term. The versions, units, environments, consumers, services, response expectations, and time periods for which the organization accepts support responsibility.

Decision question. What is supported, for whom, until when, through which route, and what is explicitly outside the commitment?

What must be decided. Identify supported units and versions or service scope, support level, consumer/channel, maintenance and end-of-support dates, response route and expectations, dependencies on enterprise support, and treatment of experimental, mirrored, archived, or retired assets.

Why it matters. Support scope controls current-output inventory, vulnerability obligations, release evidence, rollback readiness, and lifecycle interpretation. Without it, stable low activity can look abandoned and obsolete versions can remain implicitly supported forever.

Acceptable values. Current service only; named current and maintenance release lines; time-bounded LTS; best-effort experimental support with clear limits; upstream-only mirror support; or explicitly unsupported archive/retirement with continuing security obligations stated where applicable.

> **Recommended default.** Name the currently supported service or release lines and explicit EOL dates; state that versions not listed are unsupported. Use durable team support routes and avoid indefinite implied support.

Decision authority. Business/service or product owner with support and release owners; security owner confirms any continuing vulnerability-reporting obligation after normal support ends.

Record in. Platform/industry: SUPPORT.md is a GitHub-recognized community file; release notes and repository releases may identify support lines. Put an executive summary in README.md. **RH recommendation:** lifecycle and supported tri-state in .github/repository-health.toml. **External system of record:** service catalog and support/ITSM system.

Evidence and review. Supported-version table, routes, monitoring owner, EOL dates, release lines, consumer records, service catalog, route tests, and owner approval. Review on every release-line or service-lifecycle change and at least annually for Baseline, more often when risk or obligations require.

Source basis. SRC-001; controls DCR-03, DCR-04, RRO-04, and RLP-03; SRC-018.

19. Security-reporting policy

Term. The private, monitored process through which a vulnerability can be reported, acknowledged, triaged, coordinated, and resolved for the supported scope.

Decision question. Where should a reporter send a sensitive finding, who monitors it, what response should they expect, and which versions or services are covered?

What must be decided. Define a private reporting route, supported scope/versions, monitored security owner, acknowledgement and triage expectations, requested report content, safe-harbor or disclosure guidance where approved, coordinated-disclosure process, alternate route, and route-verification frequency. Do not direct sensitive reports only to a public issue tracker.

Why it matters. A visible but unmonitored address is not a functioning security control. Clear private routing reduces public exposure, lost reports, and ambiguity about unsupported versions.

Acceptable values. GitHub private vulnerability reporting or security advisory route; monitored security email; internal security portal or service desk for private repositories; or an inherited organizational policy with a visible repository link. Public repositories need a publicly discoverable private reporting route.

> **Recommended default.** Publish SECURITY.md with supported versions/scope, GitHub private vulnerability reporting where available, a durable monitored team contact, expected acknowledgement/triage behavior, and a fallback route. Verify delivery separately from reading the file.

Decision authority. Security owner with the product/service owner and legal or communications authority for disclosure language.

Record in. Platform/industry: SECURITY.md is a GitHub-recognized community file and GitHub's private vulnerability-reporting settings are the platform record. **External system of record:** security case/vulnerability system and monitored team routing; do not copy sensitive cases into the repository.

Evidence and review. Exact SECURITY.md, supported scope, enabled private route, monitored ownership, successful route test, case timestamps, and disclosure records. Verify at least every 365/180/90 days for Baseline/Elevated/Critical and after ownership or routing changes.

Source basis. SRC-001; control DCR-03 and M-DCR-03; SRC-004 and SRC-018; GitHub's security policy guidance.

20. Rollback/withdrawal policy

Term. The version-appropriate method for containing and replacing a bad output: rollback, roll-forward, revert, restore, withdrawal, reissue, or republication.

Decision question. For each unit and failure mode, how will the organization return to a safe supported state or prevent further consumption?

What must be decided. Choose the primary and fallback recovery strategies; define prerequisites, data/schema treatment, artifact or package withdrawal, authorization, target state, maximum tolerable constraints, verification, communications, and evidence retention. State when rollback is unsafe and roll-forward or restore is required.

Why it matters. A green build does not prove that a damaging release can be contained. Different output types require different recovery actions, and an unexercised document may not work under pressure.

Acceptable values. Automated deployment rollback; controlled roll-forward hotfix; GitOps revert; database forward-fix plus restore; package yank/deprecation and corrected release; model rollback; infrastructure restore; content republication; or archive restoration where the retention contract requires it.

> **Recommended default.** Define one supported recovery path per unit, retain the last known-good immutable identity and prerequisites, assign an authorizer and operator, verify the resulting state, and exercise the path at the risk-tier interval. Prefer a safe roll-forward when stateful rollback would increase risk.

Decision authority. Service/release or operations owner with the technical owner; data, security, or business owner approves recovery constraints that affect data, access, safety, or consumers.

Record in. RH recommendation: put the human-readable procedure in a version-controlled runbook under docs/operations/ and link it from README.md or SUPPORT.md; keep production selectors in .github/repository-health-production.json. **External system of record:** deployment, incident, backup/restore, package registry, or publication system holds execution evidence.

Evidence and review. Procedure, last-known-good identity, prerequisites, permissions, latest successful exercise or real recovery, result, timing, verification, communications, and follow-up actions. Review after every exercise, real recovery, delivery architecture change, or failed validation.

Source basis. SRC-001; control RRO-02 and M-RRO-02; SRC-003 and SRC-008.

21. Recovery exercise frequency

Term. The maximum permitted interval between successful, representative demonstrations of a unit's approved recovery method.

Decision question. How often must each current deployable or publishable unit prove that its rollback, roll-forward, restore, withdrawal, or republication method works?

What must be decided. Name the recovery method for each unit, the environment and scenario that make an exercise representative, the maximum interval between successful exercises, who may declare success, and the deadline for retesting a failed exercise. A real recovery may count only when its scope, result, and lessons are retained.

Why it matters. A runbook shows intent; an exercise shows that people, permissions, backups, artifacts, and procedures can actually restore an acceptable state. Frequency must reflect consequence and rate of operational change without penalizing a stable, low-activity repository.

Acceptable values. Baseline: 365 days, Elevated: 180 days, or Critical: 90 days; a shorter legal, safety, customer, or owner-approved interval; or N/A only for a never-released experiment or a retired unit with no current deployed, published, distributed, or supported output.

> **Recommended default.** Use 365/180/90 days for Baseline/Elevated/Critical, require a successful retest after failure, and review representativeness whenever architecture, deployment, data recovery, or support boundaries materially change.

Decision authority. Service or release owner, with the operations/reliability owner. A longer interval remains nonconforming and requires a temporary waiver from the risk owner; only Enterprise Architecture may approve a standards-level threshold change.

Record in. **RH recommendation:** link a unit-specific recovery contract from .github/repository-health.toml to docs/recovery.md or the operating runbook. **External system of record:** retain exercise execution, approvals, results, and corrective actions in the incident, change, continuity, or operations platform.

Evidence and review. Retain the last successful and last attempted exercise, scenario, unit/version, participants, result, issues, and closure evidence. Review at every exercise and after a material recovery-path change.

Source basis. Repository Health control RRO-02 and measure M-RRO-02; NIST SSDF 1.1; DORA version-control capability guidance.

22. Evidence retention

Term. The period and storage conditions under which decision, change, release, deployment, security, and assessment evidence must remain retrievable and replayable.

Decision question. How long must the evidence supporting repository, change, release, deployment, security, exception, and assessment decisions remain available and replayable?

What must be decided. Set retention by evidence class and risk tier; define the event that starts the clock; identify the authoritative store, immutability and access requirements; address privacy, legal hold, and deletion; and state what remains after a repository or release reaches end of support.

Why it matters. A current configuration cannot reconstruct a past approval, build, artifact, deployment, incident, or exception. Retention that is shorter than the support or investigation horizon makes traceability and trend claims unverifiable; indiscriminate retention creates privacy and cost risk.

Acceptable values. An existing legal, records, contract, or regulatory schedule that is longer; or an approved local schedule. Reasonable starting points are: change/CI/deployment evidence for 1 year Baseline, 2 years Elevated, and 3 years Critical; release identity, provenance, and SBOM for the supported life of the release plus at least one year; and assessment, exception, appeal, and adjudication records for at least three years after closure. Git history and published standards versions may be retained indefinitely unless a governing policy requires disposition.

> **Recommended default.** Adopt the longer of the applicable enterprise schedule, the risk-tier period above, and the supported-life requirement. Store durable release evidence outside ephemeral CI logs and preserve hashes or immutable identifiers so a record can be replayed.

Decision authority. Records-management owner and risk owner, advised by security, legal/privacy, platform, and the standard owner.

Record in. **RH recommendation:** document the policy and evidence-class matrix in docs/evidence-retention.md, with its identifier and links in .github/repository-health.toml. **External system of record:** configure and evidence actual retention in the forge, CI/CD, artifact registry, deployment, security, GRC, and records platforms.

Evidence and review. Export retention settings, test retrieval of representative records, and record gaps. Review annually and when support, legal, privacy, platform, or risk obligations change.

Source basis. Repository Health Standard sections 3.5, 6.1, 6.7, and 8; NIST SSDF 1.1; SLSA 1.2 provenance; GitHub deployment-history platform documentation.

23. Assessment ownership

Term. The accountable roles that scope, perform, independently review, issue, and schedule follow-up for a repository-health assessment.

Decision question. Who scopes, performs, independently reviews, issues, and responds to a repository-health assessment?

What must be decided. Assign the assessment lead, assessor, independent reviewer, repository fact owner, risk acceptor, issuer, and next-review owner. Define independence for self-assessment, automation-produced results, disputes, Elevated/Critical repositories, and accepted risk.

Why it matters. Automation can collect and calculate evidence, but it cannot make itself accountable for scope, ambiguous applicability, conflicts, or issuance. Clear roles prevent repository owners from silently approving their own favorable interpretation or risk acceptance.

Acceptable values. A central assurance team; a federated assessor from another team; a repository self-assessment with independent review; or automated collection followed by accountable human issuance. Small organizations may combine operational roles, but the assessor and risk acceptor should be independent for Elevated and Critical repositories.

> **Recommended default.** Let automation produce an unissued diagnostic; name an assessment lead and repository fact owner; require a reviewer independent of the repository's day-to-day change path before formal issuance; and set the next review when the result is issued.

Decision authority. Enterprise Architecture defines the role model; the portfolio or engineering assurance owner assigns assessors; the assessment lead assigns the reviewer and issuer.

Record in. RH recommendation: populate `[assessment].assessor`, `[assessment].reviewer`, and `[assessment].next_review` in `.github/repository-health.toml`, using durable team or identity references. **External system of record:** keep role membership, attestations, conflicts of interest, and issuance approval in the identity, assurance, or GRC platform.

Evidence and review. Record standard version, scope, cutoff, named roles, reviewer disposition, issue date, next-review date, and any conflicts. Reconfirm at each assessment and when ownership changes.

Source basis. Repository Health Governance and Pilot Calibration sections 1 and 3; Scoring, Assurance, and Exceptions section 1; assessment-result schema.

Conditional-control decisions

24. Critical-path classification

Term. The mapping of repository paths and effective settings to material production, security, release, recovery, data, or control-system impact categories.

Decision question. Which repository paths and settings could materially affect production, security, release integrity, data recovery, or the control system itself?

What must be decided. Define the categories and path patterns treated as critical, how detected paths are reviewed, the required qualified owners and backups, and what happens when a path is unclassified. Include configuration outside Git when it can change the effective path.

Why it matters. Ownership and review rules work only when they cover the files and settings that can cause material harm. A generic repository owner is not necessarily qualified to review deployment credentials, release automation, security policy, infrastructure, or data migration changes.

Acceptable values. One or more of production/deployment, security, build/release, infrastructure, data migration/schema, dependency/workflow, governance/control configuration, or an organization-specific category; none only after a documented classification establishes that no such paths exist.

> **Recommended default.** Treat deployment workflows, privileged CI, infrastructure, security policy, dependency locks, release/provenance builders, data migrations, CODEOWNERS, and repository-health policy as critical until the accountable owner narrows the inventory with evidence. Require at least one qualified owner at Baseline and two qualified people or an approved continuity plan at Elevated/Critical.

Decision authority. Repository technical owner and maintainer lead, with security, platform, data, and service owners for their domains.

Record in. Platform/industry convention: use .github/CODEOWNERS for path-to-reviewer routing; GitHub also searches the repository root and docs/, but .github/ has precedence. **RH recommendation:** record categories, rationale, and continuity requirements in .github/repository-health.toml or a linked .github/critical-paths.yml. **External system of record:** effective ruleset and team-membership settings remain authoritative.

Evidence and review. Compare declared paths with detected privileged workflows, release files, infrastructure, and production settings; resolve every owner identity. Review quarterly, after a material path is added, and after ownership changes.

Source basis. Repository Health control OWM-02 and measure M-OWM-02; NIST SSDF 1.1; GitHub CODEOWNERS and rulesets documentation.

25. Supported release-line policy

Term. The explicit set of versions, branches, or release trains that receive maintenance, including their fix direction, support period, and end-of-life rules.

Decision question. Which versions or release lines receive fixes, for how long, and how are fixes reconciled with Main?

What must be decided. Declare the supported lines, support start and end dates, compatibility promise, release source, fix and backport direction, reconciliation SLO, EOL action, and whether multiple trains or temporary stabilization branches may coexist.

Why it matters. Branch age is not health by itself. A long-lived supported line may be legitimate, while an unlabeled branch can create hidden obligations, inconsistent fixes, and consumer expectations the team never intended to meet.

Acceptable values. current line only; current plus temporary stabilization; N and N-1; named LTS lines; multiple scheduled release trains; or no supported release for a never-released experiment or properly retired unit. Every supported-line option needs explicit EOL and fix-direction rules.

> **Recommended default.** Make the smallest honest commitment: support the current stable line only unless consumer, contract, safety, or migration needs justify parallel support. State that no unlisted line is implicitly supported; reconcile production-only fixes into Main or document an approved equivalent lineage path within an owner-approved SLO.

Decision authority. Product/service owner and release owner, with security and architecture approval for compatibility, LTS, or regulated commitments.

Record in. **Platform/industry convention:** publish supported versions in SECURITY.md and material release/EOL changes in release notes or CHANGELOG.md. **RH recommendation:** record parallel_support, release source, branch purposes, EOL rules, and reconciliation SLO in .github/repository-health.toml and docs/release-policy.md. **External system of record:** package registry and product/service catalog hold current release status.

Evidence and review. Maintain a current table of version/line, source ref, owner, support level, EOL, and replacement. Review at every release-train change and at least quarterly while multiple lines are active.

Source basis. Repository Health classification axes, controls CGD-05, DCR-04, and RLP-02; SemVer 2.0.0 when declared; GitLab Flow, Trunk Based Development, GitFlow, and Pro Git workflow references as methodology-specific guidance.

26. Dependency/SBOM policy

Term. The rules for declaring, discovering, identifying, and reconciling dependencies and binding a machine-readable bill of materials to a release.

Decision question. Which dependencies must be declared, locked, discovered, and represented in a release-linked software bill of materials?

What must be decided. Define dependency classes, direct and transitive coverage, version/digest requirements, discovery cadence, approved SBOM format/version, release linkage, storage, signing or attestation expectations, and how unresolved or unavailable components are represented.

Why it matters. Language manifests alone can omit actions, images, plugins, models, data, vendored components, and build tools. Without a release-specific inventory, security and operations cannot reliably determine which output contains a vulnerable or withdrawn component.

Acceptable values. Machine-readable manifests/locks and discovery for all direct dependencies at every tier; a release-linked SPDX SBOM for Elevated/Critical outputs; an equivalent recognized machine-readable SBOM approved by the security/supply-chain owner; or N/A only when a fresh discovery proves there are no external dependencies.

> **Recommended default.** Require complete direct-dependency inventory and lock/digest identity on Main. For each Elevated/Critical release, publish an SPDX 2.3 or 3.0.1 SBOM bound to the immutable release identity. Include privileged workflow/build dependencies and retain explicit unresolved markers instead of silently omitting them.

Decision authority. Dependency steward or technical owner, with the software-supply-chain security owner; the release owner approves the release binding.

Record in. **Platform/industry convention:** keep ecosystem manifests and lockfiles at their expected project locations and publish the SBOM with the release or artifact. **RH recommendation:** document scope and format in SECURITY.md or docs/dependency-policy.md and link it from .github/repository-health.toml. **External system of record:** artifact registry, attestation store, or SBOM platform holds released SBOMs and their immutable identities.

Evidence and review. Reconcile discovery against declared inventories on Main and the latest supported release; record coverage by dependency class. Review each release and whenever the build or dependency ecosystem changes.

Source basis. Repository Health control SSC-02 and measure M-SSC-02; SPDX 3.0.1 and 2.3; OpenSSF OSPS Baseline v2026.02.19; OpenSSF Scorecard v5.5.0; SLSA 1.2 provenance.

27. Vulnerability-remediation targets

Term. The severity- and exposure-based deadlines within which a verified vulnerability or dependency risk must receive an evidenced disposition.

Decision question. How quickly must verified vulnerabilities and material dependency risks be owned, remediated, or otherwise dispositioned?

What must be decided. Define severity and exposure model, when the clock starts, target times, owner, pause rules, proof of remediation or false positive, equivalent-control criteria, escalation, and how accepted risk is displayed after the target expires.

Why it matters. Scanning without accountable deadlines measures tool activity rather than risk reduction. Inconsistent clocks or silent exclusions make teams incomparable and allow old findings to disappear without remediation.

Acceptable values. The stricter applicable enterprise, regulatory, contract, or safety target; exposure-adjusted targets with documented criteria; or the standard's starting targets. A practical full set is Critical: 7 days, High: 30 days, Medium: 90 days, and Low: 180 days. A verified false positive or approved equivalent control may close a finding with evidence; accepted risk does not make an overdue finding conforming.

> **Recommended default.** Adopt 7/30/90/180 calendar days from the verified first-known time, with immediate escalation for actively exploited or production-exposed findings. Do not pause the clock for ownership transfer, backlog status, or a waiver.

Decision authority. Security vulnerability-management owner sets the policy; repository/product owners accept remediation work; a delegated risk acceptor decides temporary residual risk within authority.

Record in. **RH recommendation:** state targets and clock rules in SECURITY.md or a linked .github/vulnerability-policy.yml. **External system of record:** the vulnerability or GRC platform is authoritative for severity, first-known date, owner, due date, disposition, exception, and closure evidence.

Evidence and review. Reconcile fresh scan findings to owner and deadline records; report overdue numerator and denominator and any overdue Critical finding. Review quarterly and after threat, policy, or regulatory change.

Source basis. Repository Health control SSC-03 and measure M-SSC-03; NIST SSDF 1.1; OpenSSF OSPS Baseline v2026.02.19; OpenSSF Scorecard v5.5.0.

28. Secret-response policy

Term. The required containment, revocation, rotation, investigation, cleanup, and closure process for suspected or verified credential exposure.

Decision question. What must happen, and how quickly, when a credential or secret is suspected or verified in source, history, artifacts, logs, or automation?

What must be decided. Define prevention and scan coverage, verification and severity, immediate containment, revoke/rotate targets, downstream dependency handling, history/artifact remediation, notification and incident thresholds, allowlist approval/expiry, closure evidence, and post-incident review.

Why it matters. Removing text from the latest commit does not invalidate a credential or erase reachable history, caches, forks, artifacts, or logs. A revoke-first process limits harm while investigation and cleanup proceed.

Acceptable values. Immediate revocation on verification with a measured target such as within 4 hours for public or Critical exposure and within 24 hours for other verified live credentials; a stricter incident policy; or immediate emergency revocation without waiting for normal change approval. An allowlist is acceptable only for a verified non-secret pattern with owner, rationale, review, and expiry.

> **Recommended default.** Revoke or disable first, rotate dependent systems, investigate use, then clean history/artifacts when required. Target four hours for public/Critical exposure and 24 hours otherwise, and test preventive plus current-and-history detection within 365/180/90 days for Baseline/Elevated/Critical.

Decision authority. Security incident owner and credential/system owner; legal/privacy and business incident leadership join when notification thresholds are met.

Record in. **Platform/industry convention:** publish the private reporting route and high-level response expectation in SECURITY.md. **RH recommendation:** keep repository-specific scan/allowlist rules in a linked policy file. **External system of record:** retain sensitive finding detail, revocation proof, notification, and incident chronology in the incident/security platform rather than the public repository.

Evidence and review. Preserve prevention configuration, fresh current/history scan, finding state, revocation/rotation timestamps, reviewed allowlist, and incident closure. Review controls within 365/180/90 days and after every verified exposure.

Source basis. Repository Health control SSC-01 and measure M-SSC-01; NIST SSDF 1.1; OpenSSF OSPS Baseline v2026.02.19; OpenSSF Scorecard v5.5.0.

29. CI/CD trust policy

Term. The rules that determine which identities, code, inputs, dependencies, runners, credentials, and approvals may exercise privileged automation.

Decision question. Which identities, code, dependencies, runners, inputs, and approvals may exercise privileged automation or change production?

What must be decided. Define default token permissions, job-level grants, untrusted pull-request/fork isolation, executable dependency pinning, runner trust zones, secret release, protected environments, publisher identity, human approval, concurrency, provenance, and emergency access.

Why it matters. CI/CD joins source control to artifacts and production. An otherwise protected Main can be bypassed when untrusted input reaches a write token, mutable action, persistent runner, deployment credential, or unprotected environment.

Acceptable values. Read-only default permissions with narrow job grants; no secret or write-token access for untrusted code; third-party executable dependencies pinned to full immutable revisions; an approved internal action registry with equivalent immutability and review; protected environment secrets; scoped federation such as OIDC; isolated ephemeral runners; and audited break-glass access.

> **Recommended default.** Set workflow permissions to read-only, grant only the deploying job what it needs, isolate untrusted events, pin external actions and images immutably, prefer short-lived federated credentials, separate validation from deployment, protect production environments, and retain durable deployment history.

Decision authority. CI/platform owner and security owner, with the release/service owner for production approval policy.

Record in. **Platform/industry convention:** workflow files under .github/workflows/ declare job behavior; GitHub rulesets, environments, repository/organization Actions policy, secrets, and runner settings hold effective enforcement. **RH recommendation:** use .github/repository-health-evidence-policy.json to identify authoritative evidence workflows and document the trust model in SECURITY.md or docs/ci-trust.md. **External system of record:** identity, runner, cloud, and deployment platforms retain effective permissions and audit logs.

Evidence and review. Analyze exact-Main workflow files and export effective permissions, action pins, triggers, data flows, environment rules, runner identity, and deployment history. Review each privileged workflow change and at least quarterly.

Source basis. Repository Health controls SSC-04 and RRO-03; SLSA 1.2; OpenSSF OSPS Baseline v2026.02.19; OpenSSF Scorecard v5.5.0; GitHub rulesets and deployments documentation.

30. Toolchain and technical-obligation policy

Term. The owned inventory of supported runtimes, build tools, generators, platforms, material debt, and maintenance commitments with due or review dates.

Decision question. Which runtimes, build images, package managers, generators, platforms, and material technical obligations must remain supported, owned, and current?

What must be decided. Define the inventory boundary, supported-version rule, ownership, due/review dates, EOL response, update and clean-build cadence, debt materiality, exception handling, and the system that preserves quarterly trend evidence.

Why it matters. A repository can look quiet and healthy while its runtime, build image, generator, or debt obligation becomes unsupported. Conversely, low commit volume should not be mistaken for abandonment when obligations remain current.

Acceptable values. vendor-supported versions only; a documented extended-support version; a pinned reproducible toolchain with planned upgrade; or a time-bounded accepted-risk record for an

unsupported component. Suggested human review intervals are semiannual Baseline, quarterly Elevated, and monthly Critical, plus event-driven review on EOL or material vulnerability notice.

> **Recommended default.** Inventory every material tool and obligation with owner, support status, due date, and disposition; pin reproducible versions or image digests; use supported components; review semiannually at Baseline, quarterly at Elevated, and monthly at Critical; and require a clean build after material toolchain change.

Decision authority. Repository technical owner, with platform owner for shared tooling and the product/risk owner for material debt or unsupported use.

Record in. Platform/industry convention: use ecosystem lockfiles, .tool-versions or equivalent version files, and immutable build-image references where supported. **RH recommendation:** maintain docs/technical-obligations.md or .github/technical-obligations.yml. **External system of record:** link work items, lifecycle notices, and accepted risks from the issue, portfolio, vulnerability, or GRC platform.

Evidence and review. Retain a current obligation register, component support evidence, owner/due date, clean-build result, and four-quarter trend. Review on the chosen cadence and on EOL/security events.

Source basis. Repository Health control OWM-03 and measure M-OWM-03; DORA version-control capability guidance; OpenSSF Scorecard v5.5.0.

31. Generated or binary artifact disposition

Term. The approved classification and handling of generated files, opaque binaries, media, data, and other artifacts that are not directly reviewable source.

Decision question. Which generated files or binary artifacts may be committed, and what makes each retained item reviewable and trustworthy?

What must be decided. Classify detected items; decide remove, regenerate, store in a registry, use Git LFS, vendor, or retain; define necessity, authoritative source, provenance/digest, deterministic regeneration or update process, owner, review path, and exception treatment.

Why it matters. Opaque executables and generated outputs can hide malicious or stale content, make review ineffective, and let committed output drift from its source. Some media, data, vendored tools, and generated source are legitimate, so an absolute ban is also misleading.

Acceptable values. not committed; registry is authoritative; generated and reproducible from reviewed source; legitimate media/data in Git or Git LFS with ownership and provenance; vendored binary with documented necessity, verified digest, update process, and security owner; or removal. An exception alone is not a conforming disposition.

> **Recommended default.** Do not store release/build outputs or executable binaries as authoritative source. Publish outputs to an artifact registry. Retain generated source only when deterministic regeneration and diff review are practical; retain media/data with explicit classification; permit a vendored executable only as a narrowly approved last resort.

Decision authority. Repository technical owner; security approves executables; data, content, legal, or records owners approve their specialized artifacts.

Record in. **Platform/industry convention:** use .gitignore for build outputs and .gitattributes for Git LFS or generated-file treatment. **RH recommendation:** document retained-item classifications in docs/generated-artifacts.md or .github/repository-artifacts.yml. **External system of record:** artifact registry or provenance store holds released binaries and immutable digests.

Evidence and review. Run exact-revision detection; map every item to classification, necessity, digest/provenance, regeneration/update process, and owner. Review each new class and at least annually.

Source basis. Repository Health control OWM-04 and measure M-OWM-04; OpenSSF OSPS Baseline v2026.02.19; OpenSSF Scorecard v5.5.0.

32. Public-contribution policy

Term. The published contract stating whether and how people outside the maintaining organization may propose issues or changes and what expectations apply.

Decision question. May the public propose issues, patches, or pull requests, and under what legal, behavioral, review, and support expectations?

What must be decided. Choose open, limited, or closed contribution; define accepted channels, contributor prerequisites, license/CLA/DCO terms if applicable, code of conduct, review and response expectations, decision authority, security-reporting route, and whether contribution implies support.

Why it matters. A public repository is visible, but that alone does not promise that maintainers accept changes, provide support, or grant rights. Ambiguity wastes contributor effort and can create legal, safety, moderation, and security problems.

Acceptable values. open to issues and pull requests; issues only; pull requests by invitation; fork/patch workflow; or public read-only/closed to contributions. Each value must state maintainer discretion, review path, rights terms, conduct expectations, and the private vulnerability route.

> **Recommended default.** For a maintained public standard or tool, accept issues and pull requests through the controlled path, state that acceptance is discretionary and does not create a support promise, require applicable tests and review, and publish a code of conduct and private security route. Choose explicit read-only status if the organization cannot sustain triage.

Decision authority. Maintainer/product owner, with legal for contribution terms, community owner for conduct, and security for vulnerability reporting.

Record in. Platform/industry convention: place CONTRIBUTING.md in the repository root, .github/, or docs/; use CODE_OF_CONDUCT.md, issue/pull-request templates, SECURITY.md, and SUPPORT.md where applicable. **External system of record:** CLA/DCO service, moderation log, or legal record remains authoritative when used.

Evidence and review. Verify that contribution links resolve, templates match policy, response ownership is active, and enforced checks do not contradict the guide. Review annually and when contribution capacity or legal terms change.

Source basis. Repository Health controls DCR-02 and DCR-03; GitHub healthy-contribution and community-profile documentation; OpenSSF OSPS Baseline v2026.02.19.

33. DORA applicability

Term. The determination that a repository maps to an operated product or service with reliable delivery and incident data suitable for a separate outcome panel.

Decision question. Does this repository map to an operated product or service with reliable delivery and incident data suitable for a separate DORA outcome panel?

What must be decided. Identify the service/unit boundary, production environment, data sources, observation window, metric formulas, coverage limitations, and owner; or record why the panel is unavailable or inapplicable. Decide whether a monorepo needs multiple panels.

Why it matters. DORA measures delivery outcomes, not repository-control conformance. Applying service metrics to documentation, templates, archives, or mixed monorepos can mislead leadership; omitting reliable service outcomes can hide whether repository practices improve delivery.

Acceptable values. available for a named service/unit with complete deploy and recovery/failure data; unavailable with a specific data-access or quality reason; or not applicable for a repository with no operated service or meaningful deployment outcome. Multiple units may have separate panels.

> **Recommended default.** Enable DORA only for a directly mapped operated service or deployable unit with trustworthy event data. Report the five current measures—change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate—as informative_only: true; never blend them into the 0–100 score.

Decision authority. Service/product owner and assessment lead, with the delivery-analytics or reliability owner validating source quality.

Record in. RH recommendation: declare availability, reason, service/unit, period, evidence IDs, context, and limitations under [dora] in .github/repository-health.toml. **External system of record:** deployment,

incident, observability, and analytics platforms remain authoritative for events and calculations; the assessment stores references and the resulting panel.

Evidence and review. Reconcile metric populations to the named service and period; disclose gaps and formula changes. Review each assessment and after delivery-system or service-boundary change.

Source basis. Repository Health Scoring, Assurance, and Exceptions section 9 and DORA metric schema; DORA software-delivery performance metrics and metrics-history guidance.

34. Portfolio authority

Term. The governed system and accountable owner that establish canonical repository identity, ownership, lifecycle, risk, units, and relationships across the portfolio.

Decision question. Which system and accountable owner determine the repository's authoritative identity, lifecycle, risk, units, relationships, and organizational ownership?

What must be decided. Name the portfolio system of record, required fields, identifier, owner, confirmation cadence, conflict precedence, discovery scope for duplicates/forks/mirrors, and the process for registering, transferring, or retiring a repository.

Why it matters. Repository-local declarations can become stale or self-serving. A portfolio authority prevents orphaned repositories, conflicting sources of truth, deployment from undeclared copies, and inconsistent type or risk decisions.

Acceptable values. An enterprise service/software catalog; a governed organization repository inventory; a CMDB/product registry; or, for a small organization, a version-controlled central inventory with an accountable portfolio owner. A repository-only record is acceptable only when it is the formally governed portfolio record rather than a self-approval.

> **Recommended default.** Use an external service/software catalog as authority and keep a repository-level overlay for Main, units, methodology, and evidence links. Treat conflicts as `Unknown` until reconciled; do not choose whichever value improves the grade.

Decision authority. Enterprise Architecture or portfolio governance owner, with service/product and repository owners confirming facts.

Record in. **RH recommendation:** put the external portfolio identifier, authoritative repository URI, owner, type, lifecycle, risk tier, and units in `.github/repository-health.toml`. **External system of record:** the catalog/CMDB holds organizational authority, relationships, data classification, consumers, successors, and confirmation history.

Evidence and review. Resolve one authoritative URI per unit, compare repository and catalog values, and retain timestamped owner confirmation. Review at least annually and on ownership, lifecycle, risk, unit, or successor change.

Source basis. Repository Health controls RLP-01 and RLP-04; SLSA 1.2 source identity principles; Pro Git distributed-workflow guidance.

35. Archive/retirement disposition

Term. The controlled end state that preserves required history and guidance while disabling or removing unsupported change, release, credential, and automation paths.

Decision question. What must be preserved, disabled, redirected, or removed when active change, release, support, or operation ends?

What must be decided. Choose archive, retirement, read-only mirror, transfer, or deletion; set effective and security/support EOL dates; identify successor and migration; inventory consumers; preserve last supported release and required evidence; define retention; revoke credentials; and disable or justify automation, webhooks, deployments, and package publication.

Why it matters. Unexplained inactivity looks like abandonment, while an archived repository with live credentials or automation can still change production. Deletion can destroy lineage, rights, audit, or restoration evidence that consumers need.

Acceptable values. archived/read-only with retained history; retired tombstone with successor; maintained read-only mirror; transferred to a new authority; or deleted after records/legal and consumer approval. Active is not a retirement disposition and is N/A for this decision.

> **Recommended default.** Prefer read-only archival over deletion: publish a dated notice and successor, preserve the last supported release and lineage, set support/security EOL, protect retained refs, revoke credentials, disable deployment and unintended automation, and update the portfolio record.

Decision authority. Product/service and portfolio owners, with security, records/legal, repository administration, and known consumer representatives as applicable.

Record in. **Platform/industry convention:** display status and successor prominently in README.md, use the forge's archive/read-only setting, and update SECURITY.md and SUPPORT.md. **RH recommendation:** set lifecycle and disposition links in .github/repository-health.toml. **External system of record:** portfolio, identity, package/deployment, records, and consumer-management systems hold closure evidence.

Evidence and review. Record disposition date, successor/migration, consumer discovery, last release, EOL, retention, ref protection, credentials, automation, and owner. Verify at closure and on the retention schedule.

Source basis. Repository Health control RLP-03 and measure M-RLP-03; OpenSSF OSPS Baseline v2026.02.19; GitHub healthy-contribution/community documentation for visible status and routes.

Formal-disposition decisions

36. N/A determination

Term. An evidenced and approved determination that a control or gate's stated applicability trigger is genuinely absent for the assessed scope.

Decision question. Is a control or gate genuinely not triggered by the repository's real type, lifecycle, units, or supported outcomes?

What must be decided. Identify the exact control/gate and affected unit, cite its applicability rule, state which triggering condition is absent, provide evidence, name approver and dates, and set a review trigger. Determine each unit separately when applicability differs.

Why it matters. N/A removes a control from scoring. Using it for missing evidence, lack of access, difficulty, cost, or current incapability inflates the score and hides risk; those conditions are `Unknown` or nonconforming.

Acceptable values. `N/A` only when the catalog's stated condition is false and evidence supports that fact; item-level N/A within a composite measure; or `applicable`. Examples include no external dependencies after discovery, no deployment/publication, or a retired unsupported unit under its type-specific rule. `Unknown` is required when the decision cannot be evidenced.

> **Recommended default.** Keep the control applicable unless the assessor can cite the exact N/A rule and independent evidence. Make N/A narrow, unit-specific, approved, and reviewable; never infer it from an empty event population alone.

Decision authority. Assessor proposes and the assessment lead or independent reviewer approves with the relevant control-domain owner; Enterprise Architecture decides a novel interpretation that could create precedent.

Record in. RH recommendation: store the control override and rationale in `.github/repository-health.toml` with `n_a_approved_by`, `n_a_approved_at`, and `n_a_review_date`, or in a linked decision register; reproduce it in the assessment result. **External system of record:** link approval when enterprise governance requires it.

Evidence and review. Retain applicability rule, factual evidence, scope, approver, decision date, review date, and changed-condition trigger. Revalidate every assessment and whenever type, lifecycle, unit, or output changes.

Source basis. Repository Health Standard section 5, Scoring, Assurance, and Exceptions section 7.1, control-specific N/A rules, and assessment schema.

37. Equivalent-control approval

Term. An approved demonstration that an alternate mechanism provides prevention, detection, traceability, and evidence equal to or stronger than the original control.

Decision question. Does an alternate mechanism achieve the original control outcome with equal or stronger prevention, detection, and traceability?

What must be decided. State the original outcome, alternate mechanism and scope, comparison criteria, evidence, residual gaps, owner, approval, effective date, review trigger, and whether the equivalence is reusable or repository-specific.

Why it matters. Methodology neutrality requires different implementations to receive equal credit when outcomes are equivalent. Calling a weaker or merely different practice "equivalent" would turn flexibility into an untracked exception.

Acceptable values. A pattern already recognized by the control catalog or approved methodology profile; a case-specific equivalent with evidence and Architecture approval; or `not equivalent`, leaving the observed control rating in place. An equivalent control is not a waiver and, when proven, is rated normally.

> **Recommended default.** Use an already recognized pattern when available. For a novel mechanism, require an explicit side-by-side outcome analysis and E3/E4 evidence proportionate to the original control; approve it narrowly before considering it a reusable precedent.

Decision authority. Control-domain owner evaluates; Enterprise Architecture approves any equivalence not already recognized in a methodology profile. Security/risk owners concur when their domain is affected.

Record in. RH recommendation: create a versioned ADR under `docs/decisions/` and a machine-readable `Equivalent control` entry in the assessment decision register or `.github/repository-health.toml`. Publish reusable, non-sensitive precedent through the standard's governance record.

Evidence and review. Preserve original requirement, alternate design, scope, prevention/detection/traceability comparison, demonstrations, limitations, approvers, and dates. Review on material mechanism or standard-version change.

Source basis. Repository Health Scoring, Assurance, and Exceptions section 7.3; Governance and Pilot Calibration sections 2 and 3; methodology-neutrality acceptance criterion.

38. Temporary waiver

Term. A time-bounded approval to accept an unmet requirement while remediation proceeds, without changing the observed rating or lifting a gate cap.

Decision question. May an unmet requirement be accepted temporarily while a defined remediation plan is completed?

What must be decided. Identify scope, control and standard version, current evidence, risk, business need, compensating controls, remediation milestones, owner, risk acceptor, Architecture disposition, effective/review/expiry dates, and renewal or exit criteria.

Why it matters. A waiver makes accepted residual risk visible and time-bound. It must not make the control Met, erase a foundational gate cap, become an evergreen approval, or obscure lack of progress.

Acceptable values. Proposed, Active, Expired, Revoked, or Closed; a default duration of 90 calendar days; up to 180 days for Baseline only when justified; and no more than 90 days for Elevated or Critical. Shorter periods are appropriate when risk or remediation warrants them.

> **Recommended default.** Use 90 days or less, require an accountable milestone plan and current compensating control, and expire automatically. Renewal requires updated evidence, progress, residual-risk review, and fresh approval; an expired waiver provides no assessment treatment.

Decision authority. Named risk acceptor within delegated authority, with Architecture disposition and affected control-domain review. Higher-risk or cross-enterprise exceptions follow enterprise risk policy.

Record in. RH recommendation: add a Temporary waiver record under [[exceptions]] in .github/repository-health.toml or a linked .github/repository-health-decisions.yml, and show it beside the finding in every report. **External system of record:** GRC/risk or issue platform holds approval, milestones, sensitive rationale, and renewal history.

Evidence and review. Retain all required exception fields, control rating, assurance, approvals, milestone progress, days to expiry, and closure evidence. Review before expiry and at every assessment.

Source basis. Repository Health Standard section 8; Scoring, Assurance, and Exceptions section 8; assessment decision schema.

39. Accepted vulnerability or operational risk

Term. An accountable business decision to carry a documented residual vulnerability or operational exposure without treating it as technical closure or conformance.

Decision question. Who may accept a known vulnerability or operational exposure, for what scope and period, and with what residual-risk evidence?

What must be decided. Identify the finding, affected asset/unit and parties, threat or failure, likelihood, impact, existing and compensating controls, remediation alternative, business rationale, acceptor authority, review/expiry, escalation threshold, and relationship to any control waiver.

Why it matters. Risk acceptance is a business decision, not technical closure. It does not prove the repository is healthy, erase an overdue finding, satisfy remediation, or lift a grade cap; separating it from conformance preserves an honest standing while allowing accountable tradeoffs.

Acceptable values. remediate; close as evidenced false positive; close through approved equivalent control; accept temporarily; transfer; or avoid/retire. Permanent acceptance is appropriate only when an enterprise risk policy expressly allows it and mandates periodic review; it still does not change the observed repository-control result.

> **Recommended default.** Do not accept actively exploited, exposed Critical, live-secret, or uncontrolled-production-path risk without executive escalation and immediate containment. Time-bound other acceptance—normally no more than 90 days for High/Critical and one year for lower risk—and tie it to an owner, review, and exit condition.

Decision authority. Business/service risk owner within delegated limits, with security or operations concurrence; enterprise risk or executive authority for exposures above delegation.

Record in. **External system of record:** GRC, vulnerability, incident, or operational-risk platform is authoritative. **RH recommendation:** keep only the decision ID, scope, status, expiry, and non-sensitive summary in the repository decision register and assessment report; do not publish exploit-sensitive details.

Evidence and review. Preserve current finding evidence, residual-risk analysis, compensating controls, approval authority, dates, monitoring, and remediation/closure evidence. Review before expiry, on exposure change, and at each assessment.

Source basis. Repository Health control SSC-03 and Scoring, Assurance, and Exceptions section 8; NIST SSDF 1.1. The distinction between accepted risk and conformance is normative in this standard.

40. Assessment appeal or interpretation

Term. The governed route for challenging assessment facts, calculations, applicability, evidence interpretation, or standards ambiguity while preserving the original record.

Decision question. How may an owner challenge an assessment fact, calculation, applicability decision, evidence interpretation, or standards ambiguity?

What must be decided. Define eligible grounds, submission content and deadline, evidence cutoff treatment, who corrects facts/arithmetic, who adjudicates interpretation, escalation for precedent or policy change, preservation of original and revised results, and communication of the decision.

Why it matters. A transparent appeal route improves accuracy without allowing an unfavorable result to be negotiated away. Preserving both versions and precedent prevents silent retroactive changes and inconsistent rulings across repositories.

Acceptable values. factual/arithmetic correction; rating upheld; rating changed on existing evidence; new assessment required for evidence after cutoff;; repository-specific interpretation; reusable precedent; or standards defect/change proposal. Disagreement alone is not evidence.

> **Recommended default.** Use the existing staged path: assessment lead corrects facts/arithmetic; original assessors reconsider against the same version and cutoff; adjudication chair and domain owner resolve interpretation; Enterprise Architecture decides any new equivalence, threshold, cross-profile precedent, or policy change.

Decision authority. Assessment lead for factual correction, adjudication chair for case interpretation, and Enterprise Architecture for normative precedent or change.

Record in. RH recommendation: preserve the original assessment, appeal submission, adjudicated result, rationale, date, and precedent ID; publish non-sensitive reusable precedents under docs/decisions/ or the standards decision log. **External system of record:** assurance/GRC workflow may hold restricted evidence and approvals.

Evidence and review. Record disputed item, grounds, evidence, both original and final ratings, roles, standard version, cutoff, rationale, and downstream impact. Review precedents on standard-version change.

Source basis. Repository Health Governance and Pilot Calibration section 3 and section 6; normative-document precedence rule; assessment guide section 8.

41. Publication/issuance decision

Term. The accountable determination that an assessment is ready to be formally released to a defined audience with its evidence, limitations, authority, and review date.

Decision question. Is an assessment ready to be issued, to whom, with what authority, limitations, and follow-up date?

What must be decided. Choose draft versus issued status, audience and confidentiality, issuer and reviewer, effective date, next review, standard version, evidence cutoff, unresolved conflicts, assurance disclosure, gate caps, exception visibility, DORA separation, and whether a badge or leadership summary may be published.

Why it matters. A generated score is not automatically an authoritative assessment. Issuing an incomplete or self-approved result can mislead leadership, consumers, auditors, or the public—especially while the standard is provisional and evidence remains Unknown.

Acceptable values. diagnostic draft—not issued; issued for controlled internal trial; issued with stated limitations; withheld pending conflict/review; or, after policy permits, publicly issued. A score or badge without the supporting raw evidence, dimensions, gates, assurance, and limitations is not acceptable.

> **Recommended default.** For 0.1.0-draft, issue only for controlled trial use after independent review, with provisional status prominent. Withhold formal issuance when a conflict could change applicability, a gate, a rating, assurance, or score; require reviewer and next-review date; publish the leadership summary only with a link to the full result.

Decision authority. Assessment lead and authorized issuer; Enterprise Architecture owns publication of the standard and any enterprise-grade interpretation. Communications, legal/privacy, or security approve external disclosure when applicable.

Record in. RH recommendation: store assessor, reviewer, evidence cutoff, issue date, next review, status, limitations, and audience in the assessment JSON/report and corresponding `.github/repository-health.toml` fields. **Platform/industry convention:** immutable release or artifact publication may carry the issued report and digest. **External system of record:** assurance/GRC approval establishes issuance authority.

Evidence and review. Confirm schema completeness, independent review, calculation replay, source/evidence links, raw results, caps, exceptions, disclosures, and distribution approval. Review on the stated date or after material repository, evidence, or standard change.

Source basis. Repository Health Standard normative precedence and reporting principles; Scoring, Assurance, and Exceptions sections 1 and 10; Governance and Pilot Calibration sections 1–3; leadership-summary and assessment schemas.

Appendix A. Repository recording map

The labels below distinguish an established host/ecosystem location from a location introduced by this Repository Health (RH) standard. A platform convention describes where a host recognizes a file; it does not make the file complete or enforced. An external system remains authoritative for facts that Git cannot reliably enforce or retain.

Information	Recommended record	Basis and boundary
Purpose, status, consumers, basic use, validation entry point	`README.md`	**Platform/industry convention.** Keep the summary close to the code and link controlled detail rather than duplicating it.

Information	Recommended record	Basis and boundary
Governance roles and decision rights	GOVERNANCE.md	**Platform/industry convention.** GitHub recognizes governance as a community-health file; team membership and approval authority still come from the identity/GRC system.
Support scope, routes, and response expectations	SUPPORT.md	**Platform/industry convention.** The service desk or support platform is the **External system of record** for cases and monitored ownership.
Security reporting, supported versions, high-level response policy	SECURITY.md	**Platform/industry convention.** Keep sensitive incidents and vulnerability detail in the security/GRC **External system of record**.
Contribution and workflow expectations	CONTRIBUTING.md	**Platform/industry convention.** GitHub searches the root, .github/, and docs/; use one authoritative copy and links.
Path-based ownership and review routing	.github/CODEOWNERS	**Platform/industry convention.** GitHub also searches root and docs/; .github/ has precedence. Effective teams and rulesets are external host settings.
Rights	LICENSE, LICENSES/, and third-party notices as applicable	**Platform/industry convention.** GitHub requires a license in each repository for community-profile recognition; legal authority remains external.

Information	Recommended record	Basis and boundary
Material changes and release notes	CHANGELOG.md and immutable release records	**Platform/industry convention.** Registry/release records are the **External system of record** for published identity.
Main role, type, lifecycle, risk, owner, methodology, units, checks, assessment roles	.github/repository-health.toml	**RH recommendation.** This is the machine-readable repository declaration; it describes policy/applicability and does not prove conformance.
Current production or publication identity	.github/repository-health-production.json	**RH recommendation.** The deployment, registry, GitOps, or publication platform remains the **External system of record** and must independently corroborate the declaration.
Authoritative workflow-evidence selection	.github/repository-health-evidence-policy.json	**RH recommendation.** It identifies accepted statement producers and exact selection policy; workflow runs and artifacts remain platform evidence.
Critical-path categories beyond CODEOWNERS	.github/critical-paths.yml or a linked section in the RH declaration	**RH recommendation.** Use only if the TOML schema does not yet carry the required detail.
Vulnerability, dependency, CI trust, and secret policy overlays	SECURITY.md, docs/, or a controlled .github/*-policy.yml	**RH recommendation** for the exact overlay filename; machine-readable values should link to the enterprise security policy rather than fork it.

Information	Recommended record	Basis and boundary
Technical obligations and generated/binary dispositions	docs/technical-obligations.md, docs/generated-artifacts.md, or machine-readable .github/ registers	**RH recommendation.** Issues, lifecycle feeds, artifact registries, and accepted-risk systems remain authoritative.
N/A, equivalent-control, and temporary-waiver decisions	[[exceptions]] in the RH declaration or .github/repository-health-decisions.yml; ADR under docs/decisions/	**RH recommendation.** GRC/risk approval is the **External system of record** when enterprise policy requires it.
Rulesets, environments, secrets, runners, bypass lists, and deployment protection	GitHub organization/repository settings	**Platform/industry convention and External system of record.** GitHub documents the settings surface and its effective configuration is authoritative. A versioned export may support review, but an export does not enforce the setting.
Portfolio identity, ownership, consumers, relationships, lifecycle, and risk authority	Service/software catalog, CMDB, or governed inventory	**External system of record.** Store its stable ID and non-conflicting repository overlay in the RH declaration.
Assessment JSON, detailed report, and leadership summary	Stable report set described by the assessment schema	**RH recommendation.** An assurance/GRC approval establishes whether a generated artifact is formally issued.

Appendix B. Primary source register

The repository's controlled source register pins versions and explains whether each source is normative, conditional, authoritative platform guidance, or informative. The compact list below identifies the primary sources most relevant to decisions 21–41; it does not silently add their full contents to the RH standard.

Source	Pinned/use status	Decisions principally informed
Repository Health controlled standard, control catalog, measurement dictionary, scoring policy, and governance guide	Provisional 0.1.0-draft; RH normative/supporting roles are defined in the source register	All decisions; especially 21–41 disposition and issuance rules
NIST SP 800-218 SSDF 1.1	Final 2022; informative framework in RH 0.1.0-draft	21–22, 24, 27–29, 39
OpenSSF OSPS Baseline v2026.02.19	Current pinned baseline; conditional when conformance is claimed	26–29, 31–32, 35
SLSA 1.2 and provenance	Approved 1.2; conditional for SLSA claims and authoritative supply-chain guidance otherwise	22, 26, 29, 34
OpenSSF Scorecard v5.5.0 checks	Informative heuristic, never a substitute for control evidence	26–31
DORA software-delivery performance metrics, metrics history, and version-control capability	Informative outcome/capability guidance; DORA remains outside the RH score	21, 30, 33
GitHub rulesets and deployments	Authoritative platform references; living documentation snapshot	24, 29, 41
GitHub CODEOWNERS	Authoritative platform recording convention	24
GitHub contribution guidelines, default community-health files, and community profiles	Authoritative platform/community recording conventions; living documentation snapshot	25, 28, 32, 35

Source	Pinned/use status	Decisions principally informed
SPDX 3.0.1 and SPDX 2.3	Conditional syntax for an SPDX claim; RH release builder currently declares SPDX 2.3 JSON	26
Semantic Versioning 2.0.0	Conditional only when adopted for a public API	25
OpenGitOps Principles v1.0.0 and Pro Git distributed workflows	Authoritative methodology references; not universal workflow mandates	25, 29, 34

Appendix C. Blank decision-record template

Copy this template for a repository-specific decision. Use a stable identifier; record Unknown rather than guessing; and link sensitive evidence instead of copying it into a public repository.

DEC-XXX — Term

Record field	Value
Status	Proposed / Approved / Superseded / Expired / Revoked / Closed
Repository and units	
Standard version	0.1.0-draft
Risk tier and lifecycle	
Decision owner	
Approver(s)	
Effective date	
Review date	
Expiry or change trigger	

Record field	Value
External record ID	

Decision question.

What must be decided.

Why it matters.

Acceptable values.

> **Recommended default.**

Decision authority.

Record in. Mark each location **Platform/industry convention**, **RH recommendation**, or **External system of record**.

Evidence and review.

Source basis.

Decision selected.

Rationale and alternatives considered.

Scope, limitations, and compensating controls.

Approval and revision history.